Уточнённый план под бюджет 100к токенов

План работ по регламенту соревнования (харнесс кодового агента на базе openJiuwen): бюджет 100 000 токенов, 8 ядер, задачи по кодингу, анализу данных, математике и поиску, форматы файлов: py, csv, json, md.

Главный вывод: бюджет очень жёсткий

100 000 токенов на 100 задач дают в среднем **~1 000 токенов на задачу**, если решать все. Реально удастся решить 25–50 задач, поэтому победит тот, кто точнее выберет задачи и меньше потратит на каждую. Отсюда следуют правила:

- **Оценка расхода и выполнимости должна стоить 0 токенов.** Только локальные признаки и классические модели (эвристики, TF-IDF + GBM/LR), без LLM-зондирования. Даже 5 000 токенов на оценку — это 5% бюджета.
- **Модель по умолчанию `noreason`.** Reasoning-режим может сжечь 2–5к токенов на одну задачу. Его оставляйте для дорогих задач с высоким reward и только после оценки.
- **Не гоняйте длинные агентные циклы.** История в каждом шаге пересылается заново, поэтому расход растёт квадратично (хоть входные токены и стоят 1/10). Лучше один вызов «прочитай, напиши код/ответ», локальный запуск и максимум одна попытка исправления.
- **Системный промпт openJiuwen и описания инструментов тоже считаются.** Замерьте, сколько токенов уходит на «пустой» вызов. Если фреймворк добавляет 2–3к входных токенов (это 200–300 выходных) на каждый шаг, минимизируйте промпты и число шагов. При необходимости используйте openJiuwen тонким слоем, только для клиента и оркестрации.

Стратегия по типам задач

Тип	Подход	Где экономить
Кодинг (.py, .md)	Один вызов: условие + нужные куски файлов, на выходе патч или код. Локально запустить тесты, если они есть в каталоге задачи	Давать модели не весь файл, а релевантные функции (парсинг через `ast`)

Тип	Подход	Где экономить
Анализ данных (csv, json)	Не отправлять данные в LLM. Дать схему (колонки, типы, 3–5 строк), модель пишет pandas-код, он выполняется локально, ответ берётся из вывода	Главная экономия: код считает бесплатно, токены уходят только на сам код
Математика	`noreason` с коротким выводом, либо код (sympy/python) и локальное вычисление. Reasoning только если задача дорогая по reward	Требовать только финальный ответ, без развёрнутых рассуждений
Поиск	Выяснить, это поиск по файлам задачи или по интернету. Выход в сеть есть только через прокси к LLM, значит, скорее всего, поиск локальный (grep, BM25 по файлам). LLM нужна только для финальной формулировки	Поиск и ранжирование делать локально, без LLM

Что изменить в плане

1. Оценщик расхода (0 токенов)

Признаки: тип задачи (по расширениям файлов и ключевым словам), длина условия в символах, число и размер файлов, число колонок и строк в csv, число функций в .py, наличие тестов, наличие чисел и формул. Целевая переменная: реальные токены по вашим прогонам. Для малой выборки подойдут Ridge или GBM с монотонными ограничениями, плюс консервативный множитель.

2. Детектор невыполнимых (0 токенов)

Эвристики:

- условие ссылается на файл, которого нет;
- требует интернет или внешнее API;
- пустое или противоречивое условие;

- нет входных данных, о которых говорится в тексте;
- нужна недоступная библиотека.

Дополните классификатором на тех же признаках. Поскольку невыполнимых 15–25%, ошибка «пропустили выполнимую» обычно дешевле, чем «потратили бюджет на невыполнимую». Отсекайте агрессивнее.

3. Планировщик

Приоритет = `P(успех) × reward / E[токены]`, жадный отбор, пока не выбрано ~85% бюджета. Остальные 15% — резерв на дорогие провалы и повторы. Перепланируйте после каждой задачи по факту расхода. Сначала решайте дешёвые задачи с хорошим отношением (баллы будут, даже если бюджет внезапно кончится).

4. Жёсткие лимиты

- Лимит на задачу: 1.5–2× от оценки, затем сдача.
- `max_tokens` на каждый вызов, чтобы модель не уходила в многословие.
- Не более 1 повтора на задачу.
- Общий счётчик с блокировкой для 5 параллельных сессий. Ограничивайте число параллельных задач так, чтобы суммарные резервы не превышали остаток.

5. Выбор модели — измерениями

Сравните четыре модели на ваших примерах по метрике «баллы на 1000 токенов», а не по точности. Скорее всего, `qwen3.6-35b-a3b-noreason` окажется дешевле или быстрее, а 27b-reason пригодится точечно.

Ближайшие шаги (по порядку)

1. Написать чистый репозиторий-каркас и LLM-обёртку со счётчиком токенов (output + reasoning + input/10). Это главный измерительный прибор.
2. Прогнать все примеры задач четырьмя моделями с одним коротким промптом, записать расход и успех в таблицу.
3. По таблице решить, какая модель на какой тип, и обучить оценщик.
4. Синтезировать невыполнимые задачи (удалить файл, убрать данные из условия, подставить недостижимое требование) и обучить детектор.
5. Собрать планировщик, затем сделать прогон ровно командой из регламента с лимитом 100к.

Вопросы, которые стоит выяснить

- Как харнесс узнаёт бюджет (100к)? Его нужно принимать параметром и иметь значение по умолчанию.
- Что лежит в каталоге задачи помимо `task.json` (есть ли тесты, ожидаемый ответ в примерах)? От этого зависит, можно ли самопроверяться локально.

Пришлите по одному примеру задачи каждого типа (структуру каталога и `task.json`), и я помогу точно определить признаки для оценщика и формат промптов.